

EXP NO: 7	GENERATIVE MODELS WITH GANS: CREATING AND TRAINING A GENERATIVE ADVERSARIAL NETWORK
------------------	--

AIM:

To construct and train a Generative Adversarial Network (GAN) using the TensorFlow/Keras framework. The objective is to train the GAN on the MNIST dataset to generate new, synthetic images of handwritten digits that are indistinguishable from the original training data.

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.datasets import fashion_mnist


# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)


# -----
# 1. Load Fashion-MNIST Dataset
# -----


(x_train, _), (_, _) = fashion_mnist.load_data()

print("Original Dataset Shape:", x_train.shape)


# Use smaller dataset for faster execution
x_train = x_train[:20000]
```

```

# Normalize to [-1, 1]

x_train = x_train.astype("float32")
x_train = (x_train - 127.5) / 127.5


# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)


print("Processed Dataset Shape:", x_train.shape)


# -----
# 2. Create Dataset
# -----


BATCH_SIZE = 128


train_dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(20000)
    .batch(BATCH_SIZE)
)


# -----
# 3. Generator
# -----


LATENT_DIM = 100


def build_generator():

```

```
model = keras.Sequential()
```

```
model.add(  
    layers.Dense(  
        7 * 7 * 128,  
        use_bias=False,  
        input_shape=(LATENT_DIM,) )  
)
```

```
model.add(layers.BatchNormalization())  
model.add(layers.LeakyReLU())
```

```
model.add(layers.Reshape((7, 7, 128)))
```

```
model.add(  
    layers.Conv2DTranspose(  
        64,  
        (5, 5),  
        strides=(2, 2),  
        padding="same",  
        use_bias=False )  
)
```

```
model.add(layers.BatchNormalization())  
model.add(layers.LeakyReLU())
```

```
model.add(
    layers.Conv2DTranspose(
        32,
        (5, 5),
        strides=(2, 2),
        padding="same",
        use_bias=False
    )
)
```

```
model.add(layers.BatchNormalization())
model.add(layers.LeakyReLU())
```

```
model.add(
    layers.Conv2DTranspose(
        1,
        (5, 5),
        padding="same",
        activation="tanh"
    )
)
```

```
return model
```

```
generator = build_generator()
```

```
# -----
```

```
# 4. Discriminator
```

```
# -----
```

```
def build_discriminator():
```

```
    model = keras.Sequential()
```

```
    model.add(
```

```
        layers.Conv2D(
```

```
            64,
```

```
            (5, 5),
```

```
            strides=(2, 2),
```

```
            padding="same",
```

```
            input_shape=(28, 28, 1)
```

```
        )
```

```
    )
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Dropout(0.3))
```

```
    model.add(
```

```
        layers.Conv2D(
```

```
            128,
```

```
            (5, 5),
```

```
            strides=(2, 2),
```

```
            padding="same"
```

```
        )
```

```
)
```

```
model.add(layers.LeakyReLU())
```

```
model.add(layers.Dropout(0.3))
```

```
model.add(layers.Flatten())
```

```
model.add(layers.Dense(1, activation="sigmoid"))
```

```
return model
```

```
discriminator = build_discriminator()
```

```
# -----
```

```
# 5. Loss Functions
```

```
# -----
```

```
cross_entropy = keras.losses.BinaryCrossentropy()
```

```
def generator_loss(fake_output):
```

```
    return cross_entropy(
        tf.ones_like(fake_output),
        fake_output
    )
```

```
def discriminator_loss(real_output, fake_output):
```

```
real_loss = cross_entropy(  
    tf.ones_like(real_output),  
    real_output  
)
```

```
fake_loss = cross_entropy(  
    tf.zeros_like(fake_output),  
    fake_output  
)
```

```
return real_loss + fake_loss
```

```
# -----
```

```
# 6. Optimizers
```

```
# -----
```

```
generator_optimizer = keras.optimizers.Adam(  
    learning_rate=0.0002,  
    beta_1=0.5  
)
```

```
discriminator_optimizer = keras.optimizers.Adam(  
    learning_rate=0.0002,  
    beta_1=0.5  
)
```

```
# -----
```

7. Training Step

@tf.function

def train_step(images):

```
    noise = tf.random.normal(  
        [tf.shape(images)[0], LATENT_DIM]  
    )
```

```
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
```

```
        fake_images = generator(  
            noise,  
            training=True  
        )
```

```
        real_output = discriminator(  
            images,  
            training=True  
        )
```

```
        fake_output = discriminator(  
            fake_images,  
            training=True  
        )
```

```
    gen_loss = generator_loss(fake_output)
```



```
disc_loss = discriminator_loss(
    real_output,
    fake_output
)

gen_gradients = gen_tape.gradient(
    gen_loss,
    generator.trainable_variables
)

disc_gradients = disc_tape.gradient(
    disc_loss,
    discriminator.trainable_variables
)

generator_optimizer.apply_gradients(
    zip(
        gen_gradients,
        generator.trainable_variables
    )
)

discriminator_optimizer.apply_gradients(
    zip(
        disc_gradients,
        discriminator.trainable_variables
    )
)
```

```
)
```

```
return gen_loss, disc_loss
```

```
# -----
```

```
# 8. Generate Images
```

```
# -----
```

```
seed = tf.random.normal([16, LATENT_DIM])
```

```
def show_generated_images():
```

```
    predictions = generator(
```

```
        seed,
```

```
        training=False
```

```
    )
```

```
    predictions = (predictions + 1) / 2
```

```
    plt.figure(figsize=(6, 6))
```

```
    for i in range(16):
```

```
        plt.subplot(4, 4, i + 1)
```

```
        plt.imshow(
```

```
            predictions[i, :, :, 0],
```

```
            cmap="gray"
```

```
)
```

```
plt.axis("off")
```

```
plt.suptitle("Generated Fashion-MNIST Images")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 9. Train
```

```
# -----
```

```
EPOCHS = 10
```

```
print("Starting Fashion-MNIST GAN Training...\n")
```

```
for epoch in range(EPOCHS):
```

```
    gen_losses = []
```

```
    disc_losses = []
```

```
    for batch in train_dataset:
```

```
        gen_loss, disc_loss = train_step(batch)
```

```
        gen_losses.append(float(gen_loss))
```

```
        disc_losses.append(float(disc_loss))
```

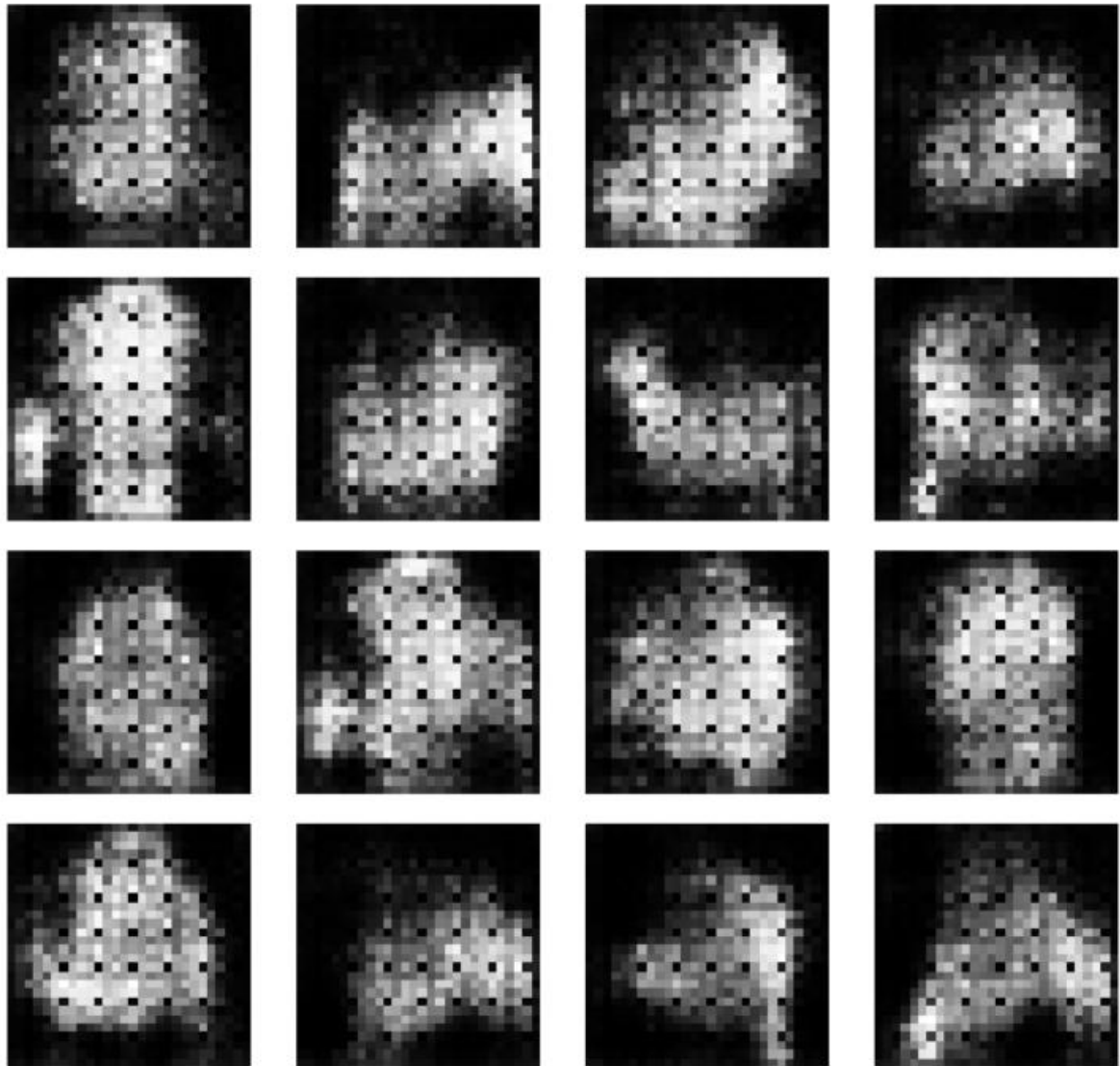
```
print(
    f"Epoch {epoch + 1}/{EPOCHS} | "
    f"Generator Loss: {np.mean(gen_losses):.4f} | "
    f"Discriminator Loss: {np.mean(disc_losses):.4f}"
)

if (epoch + 1) % 2 == 0:
    show_generated_images()

print("\nTraining Completed!")

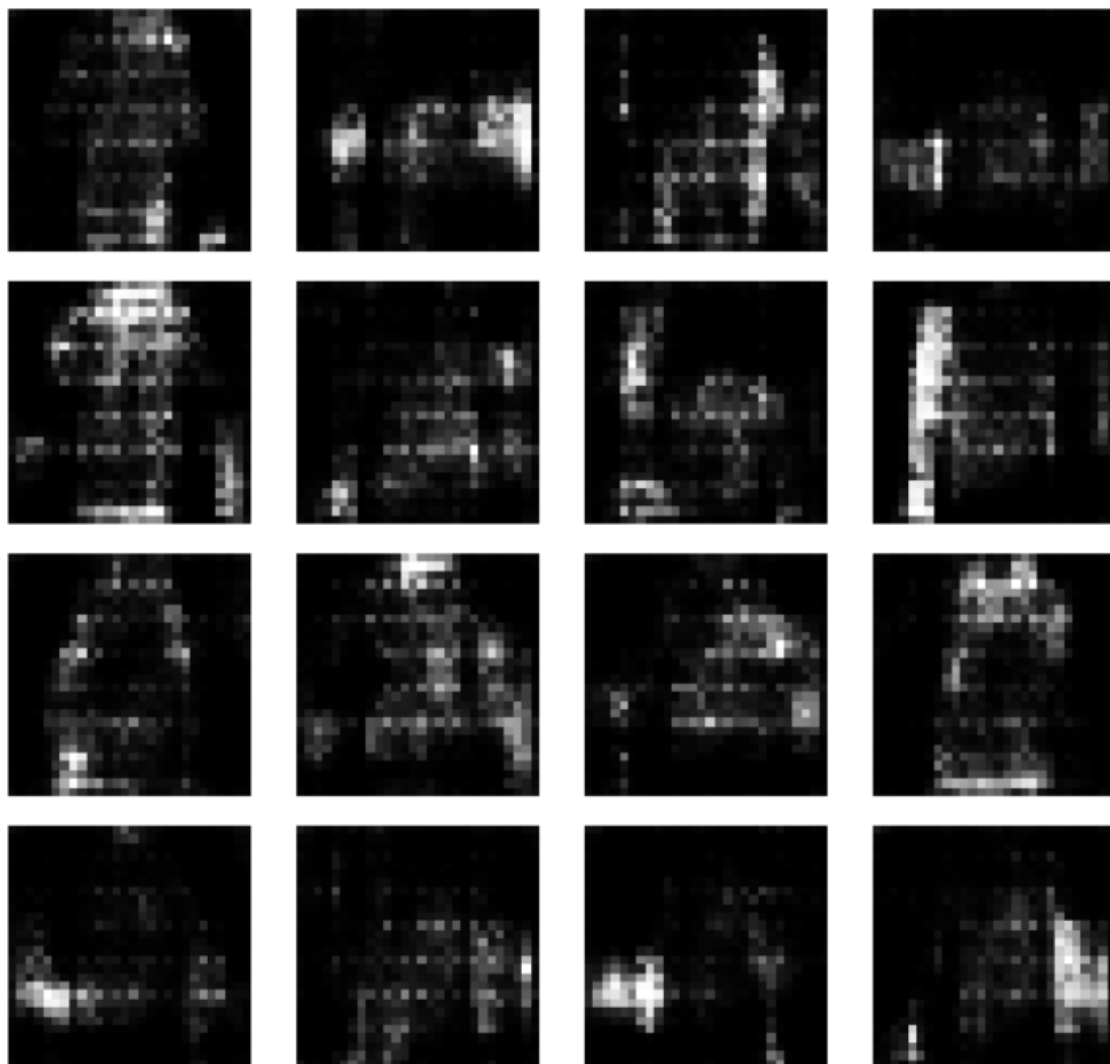
show_generated_images()
```

Generated Fashion-MNIST Images



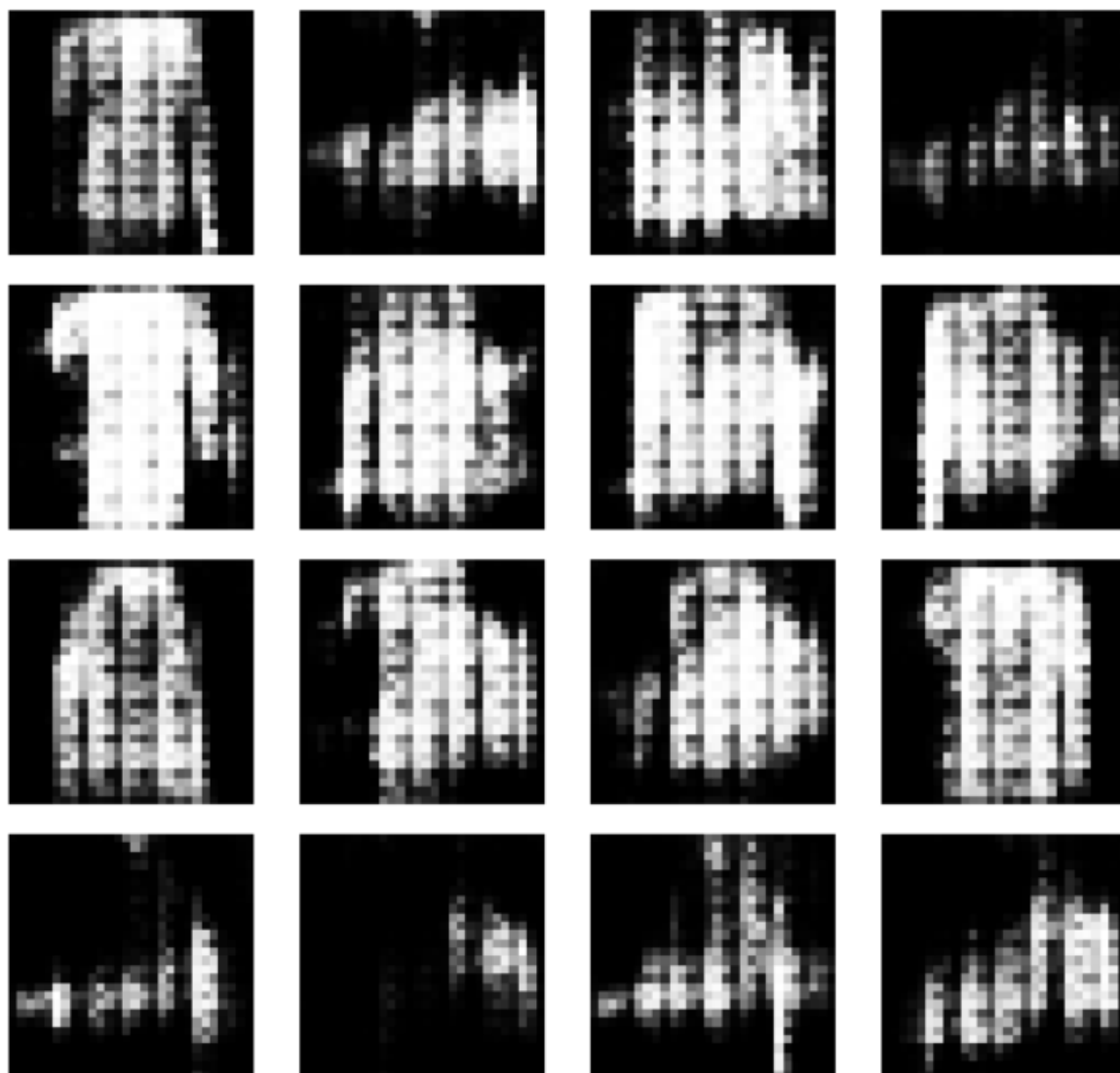
Epoch 4/10 | Generator Loss: 0.7311 | Discriminator Loss: 1.3434

Generated Fashion-MNIST Images



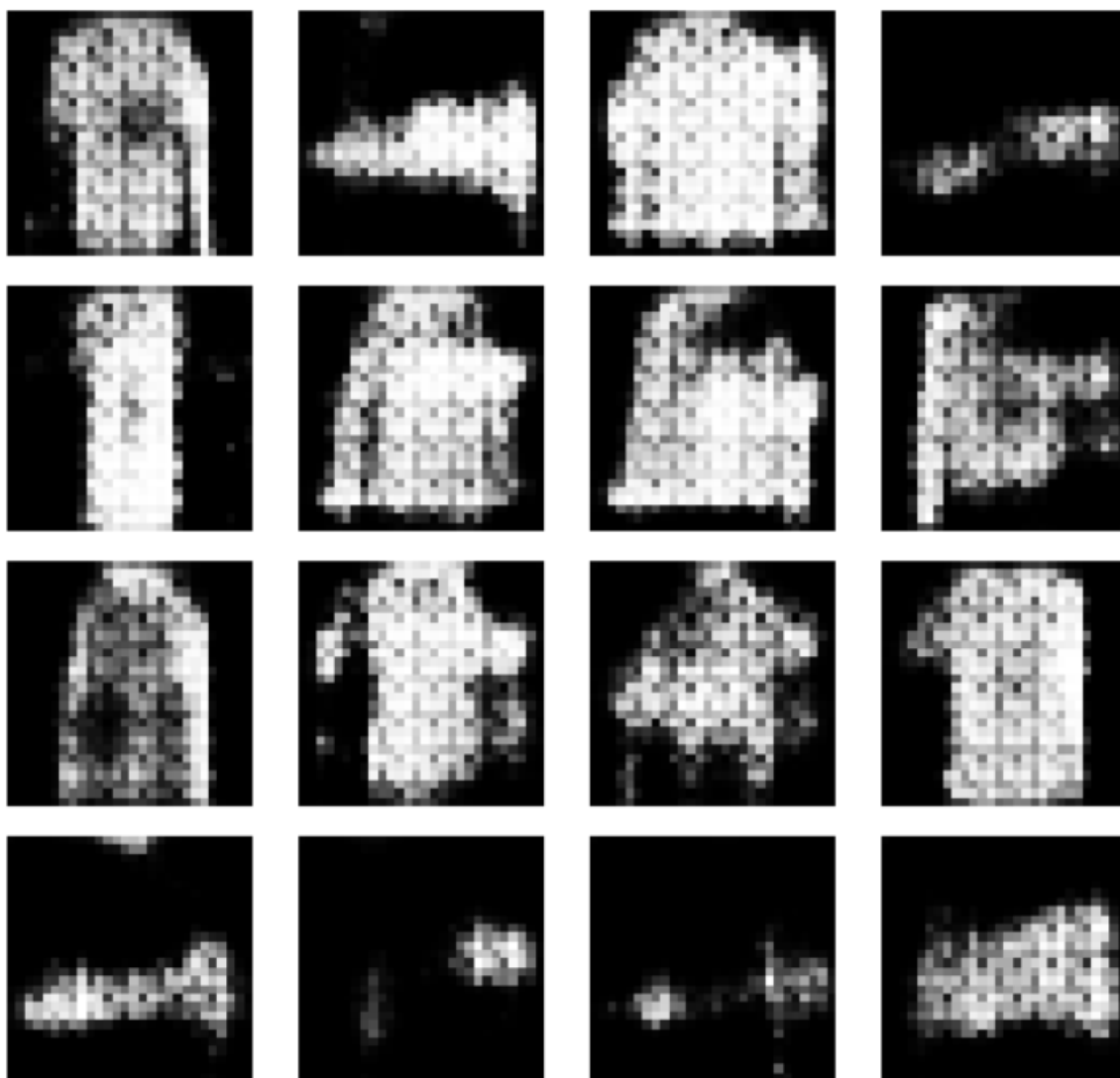
Epoch 6/10 | Generator Loss: 0.7808 | Discriminator Loss: 1.2687

Generated Fashion-MNIST Images



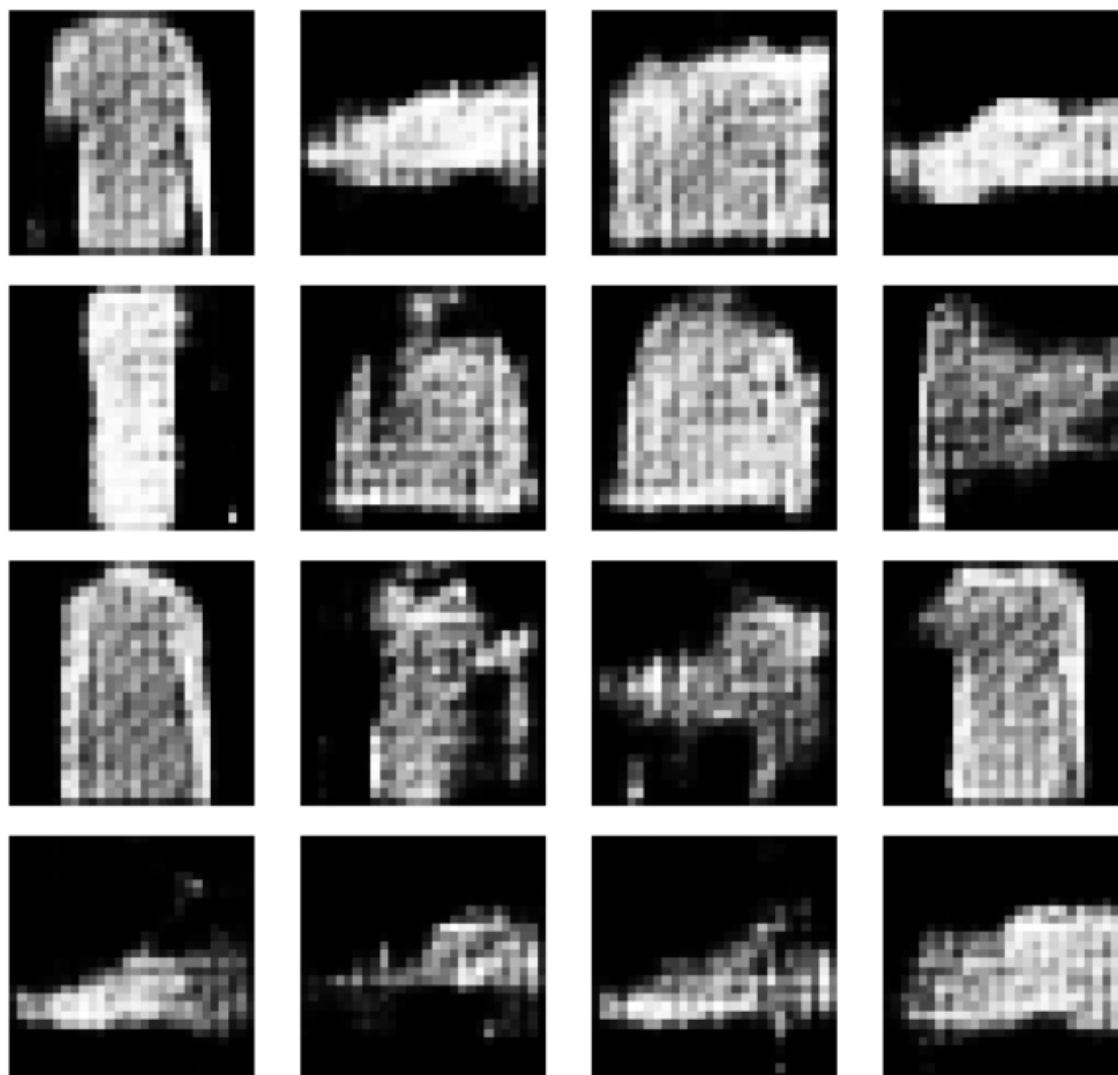
Epoch 7/10 | Generator Loss: 0.8611 | Discriminator Loss: 1.1899
Epoch 8/10 | Generator Loss: 0.9514 | Discriminator Loss: 1.1406

Generated Fashion-MNIST Images



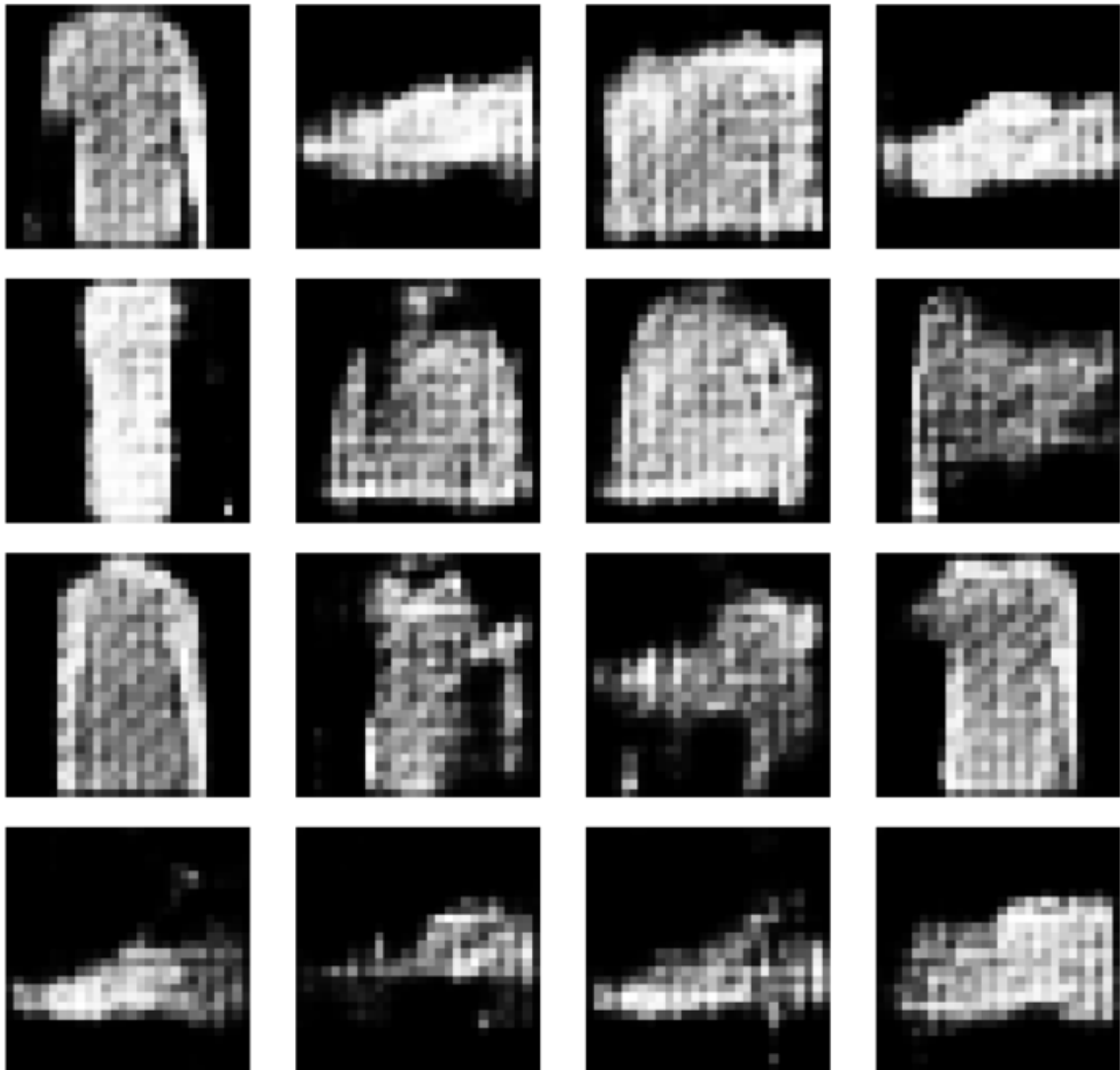
Epoch 10/10 | Generator Loss: 0.8453 | Discriminator Loss: 1.2616

Generated Fashion-MNIST Images



Training Completed!

Generated Fashion-MNIST Images



Code 2:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import fashion_mnist

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# -----
# 1. Load Fashion-MNIST Dataset
# -----

(x_train, _), (_, _) = fashion_mnist.load_data()

print("Original Dataset Shape:", x_train.shape)

# Use smaller dataset for faster execution
x_train = x_train[:20000]

# Normalize to [-1, 1]
x_train = x_train.astype("float32")
```

```

x_train = (x_train - 127.5) / 127.5

# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)

print("Processed Dataset Shape:", x_train.shape)

# -----
# 2. Create Dataset
# -----

BATCH_SIZE = 128

train_dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(20000)
    .batch(BATCH_SIZE)
)

# -----
# 3. Generator
# -----

LATENT_DIM = 100

def build_generator():

    model = keras.Sequential()

```

```
model.add(
    layers.Dense(
        7 * 7 * 128,
        use_bias=False,
        input_shape=(LATENT_DIM,)
    )
)
```

```
model.add(layers.BatchNormalization())
```

```
model.add(layers.LeakyReLU())
```

```
model.add(layers.Reshape((7, 7, 128)))
```

```
model.add(
    layers.Conv2DTranspose(
        64,
        (5, 5),
        strides=(2, 2),
        padding="same",
        use_bias=False
    )
)
```

```
model.add(layers.BatchNormalization())
```

```
model.add(layers.LeakyReLU())
```

```
model.add(
```

```
layers.Conv2DTranspose(  
    32,  
    (5, 5),  
    strides=(2, 2),  
    padding="same",  
    use_bias=False  
)  
)
```

```
model.add(layers.BatchNormalization())  
model.add(layers.LeakyReLU())
```

```
model.add(  
    layers.Conv2DTranspose(  
        1,  
        (5, 5),  
        padding="same",  
        activation="tanh"  
    )  
)
```

```
return model
```

```
generator = build_generator()
```

```
# -----
```

```
# 4. Discriminator
```

```
# -----
```

```
def build_discriminator():
```

```
    model = keras.Sequential()
```

```
    model.add(
```

```
        layers.Conv2D(
```

```
            64,
```

```
            (5, 5),
```

```
            strides=(2, 2),
```

```
            padding="same",
```

```
            input_shape=(28, 28, 1)
```

```
        )
```

```
    )
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Dropout(0.3))
```

```
    model.add(
```

```
        layers.Conv2D(
```

```
            128,
```

```
            (5, 5),
```

```
            strides=(2, 2),
```

```
            padding="same"
```

```
        )
```

```
    )
```

```
model.add(layers.LeakyReLU())
model.add(layers.Dropout(0.3))
```

```
model.add(layers.Flatten())
model.add(layers.Dense(1, activation="sigmoid"))
```

```
return model
```

```
discriminator = build_discriminator()
```

```
# -----
# 5. Loss Functions
# -----
```

```
cross_entropy = keras.losses.BinaryCrossentropy()
```

```
def generator_loss(fake_output):
```

```
    return cross_entropy(
        tf.ones_like(fake_output),
        fake_output
    )
```

```
def discriminator_loss(real_output, fake_output):
```

```
    real_loss = cross_entropy(
```



```
    tf.ones_like(real_output),  
    real_output  
)
```

```
fake_loss = cross_entropy(  
    tf.zeros_like(fake_output),  
    fake_output  
)
```

```
return real_loss + fake_loss
```

```
# -----
```

```
# 6. Optimizers
```

```
# -----
```

```
generator_optimizer = keras.optimizers.Adam(  
    learning_rate=0.0002,  
    beta_1=0.5  
)
```

```
discriminator_optimizer = keras.optimizers.Adam(  
    learning_rate=0.0002,  
    beta_1=0.5  
)
```

```
# -----
```

```
# 7. Training Step
```

```
# -----
```

```
@tf.function
def train_step(images):

    noise = tf.random.normal(
        [tf.shape(images)[0], LATENT_DIM]
    )

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        fake_images = generator(
            noise,
            training=True
        )

        real_output = discriminator(
            images,
            training=True
        )

        fake_output = discriminator(
            fake_images,
            training=True
        )

        gen_loss = generator_loss(fake_output)

        disc_loss = discriminator_loss(
```

```
        real_output,
        fake_output
    )

    gen_gradients = gen_tape.gradient(
        gen_loss,
        generator.trainable_variables
    )

    disc_gradients = disc_tape.gradient(
        disc_loss,
        discriminator.trainable_variables
    )

    generator_optimizer.apply_gradients(
        zip(
            gen_gradients,
            generator.trainable_variables
        )
    )

    discriminator_optimizer.apply_gradients(
        zip(
            disc_gradients,
            discriminator.trainable_variables
        )
    )
```

```
return gen_loss, disc_loss
```

```
# -----
```

```
# 8. Generate Images
```

```
# -----
```

```
seed = tf.random.normal([16, LATENT_DIM])
```

```
def show_generated_images():
```

```
    predictions = generator(  
        seed,  
        training=False  
    )
```

```
    predictions = (predictions + 1) / 2
```

```
    plt.figure(figsize=(6, 6))
```

```
    for i in range(16):
```

```
        plt.subplot(4, 4, i + 1)
```

```
        plt.imshow(  
            predictions[i, :, :, 0],  
            cmap="gray"  
        )
```

```
plt.axis("off")
```

```
plt.suptitle("Generated Fashion-MNIST Images")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 9. Train
```

```
# -----
```

```
EPOCHS = 10
```

```
print("Starting Fashion-MNIST GAN Training...\n")
```

```
for epoch in range(EPOCHS):
```

```
    gen_losses = []
```

```
    disc_losses = []
```

```
    for batch in train_dataset:
```

```
        gen_loss, disc_loss = train_step(batch)
```

```
        gen_losses.append(float(gen_loss))
```

```
        disc_losses.append(float(disc_loss))
```

```
    print(
```

```
        f"Epoch {epoch + 1}/{EPOCHS} | "
```

```
    f"Generator Loss: {np.mean(gen_losses):.4f} | "  
    f"Discriminator Loss: {np.mean(disc_losses):.4f}"  
)
```

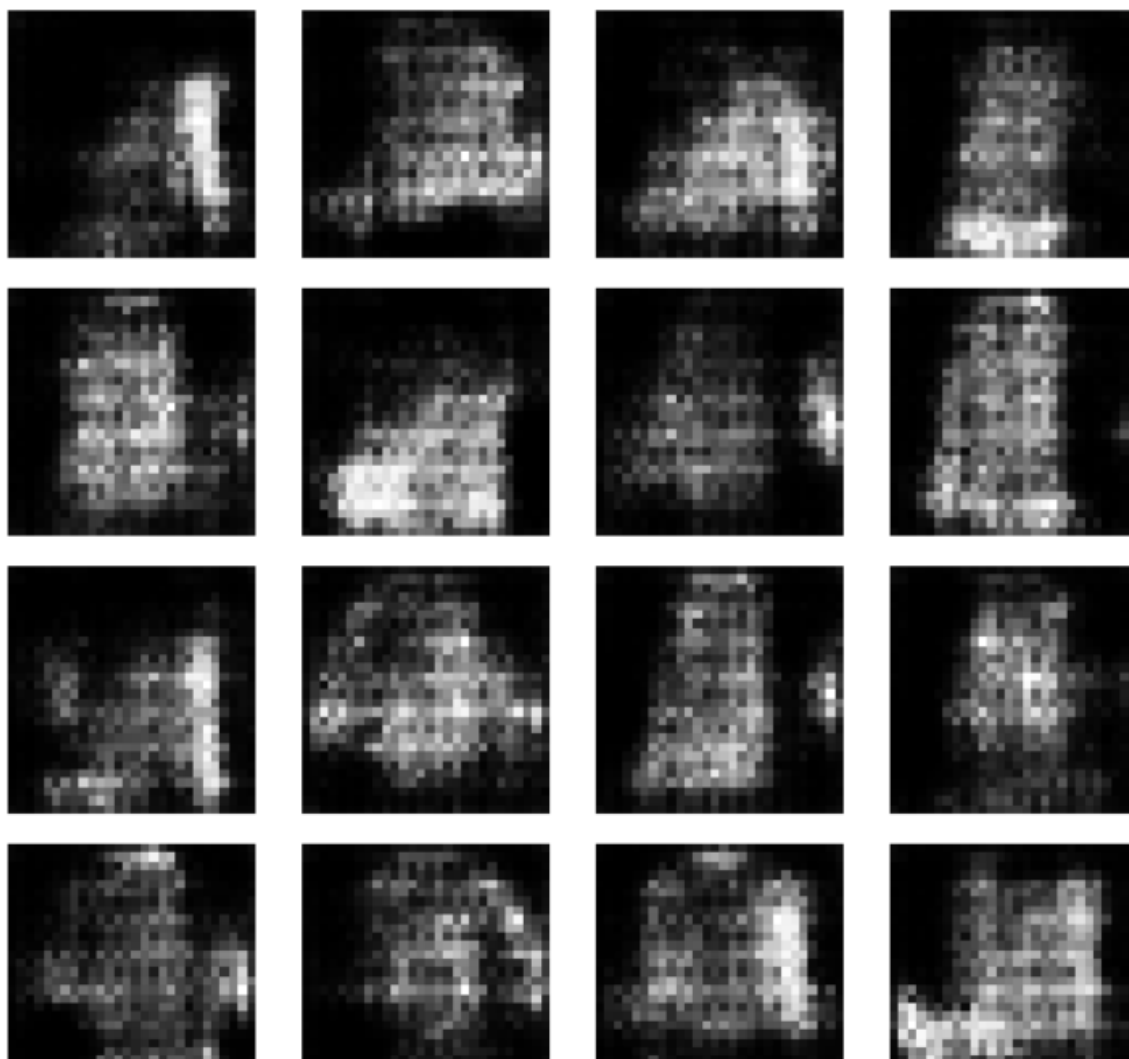
```
if (epoch + 1) % 2 == 0:  
    show_generated_images()
```

```
print("\nTraining Completed!")
```

```
show_generated_images()
```

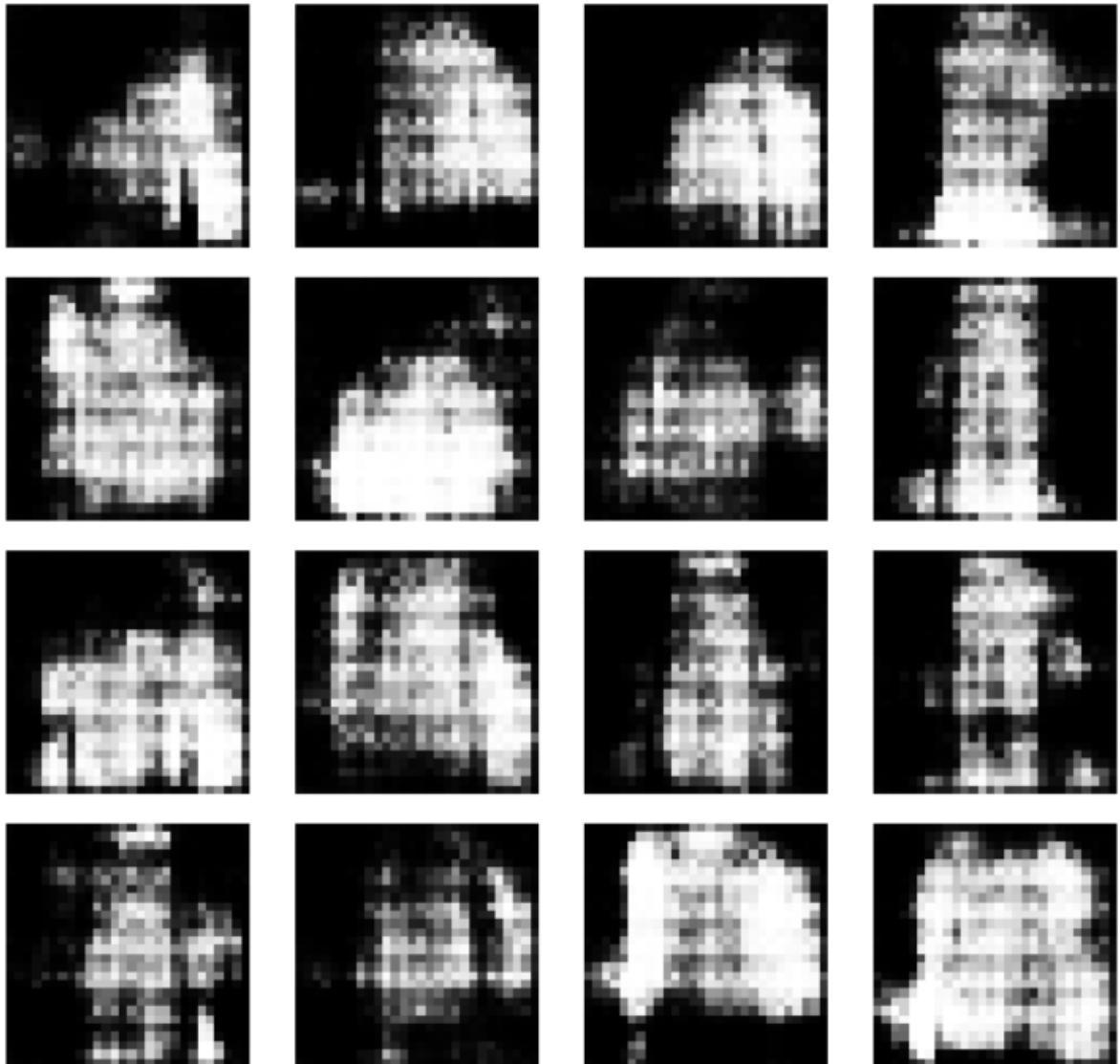
Epoch 1/10	Generator Loss: 0.7283	Discriminator Loss: 1.3093
Epoch 2/10	Generator Loss: 0.7005	Discriminator Loss: 1.3809

Generated Fashion-MNIST Images



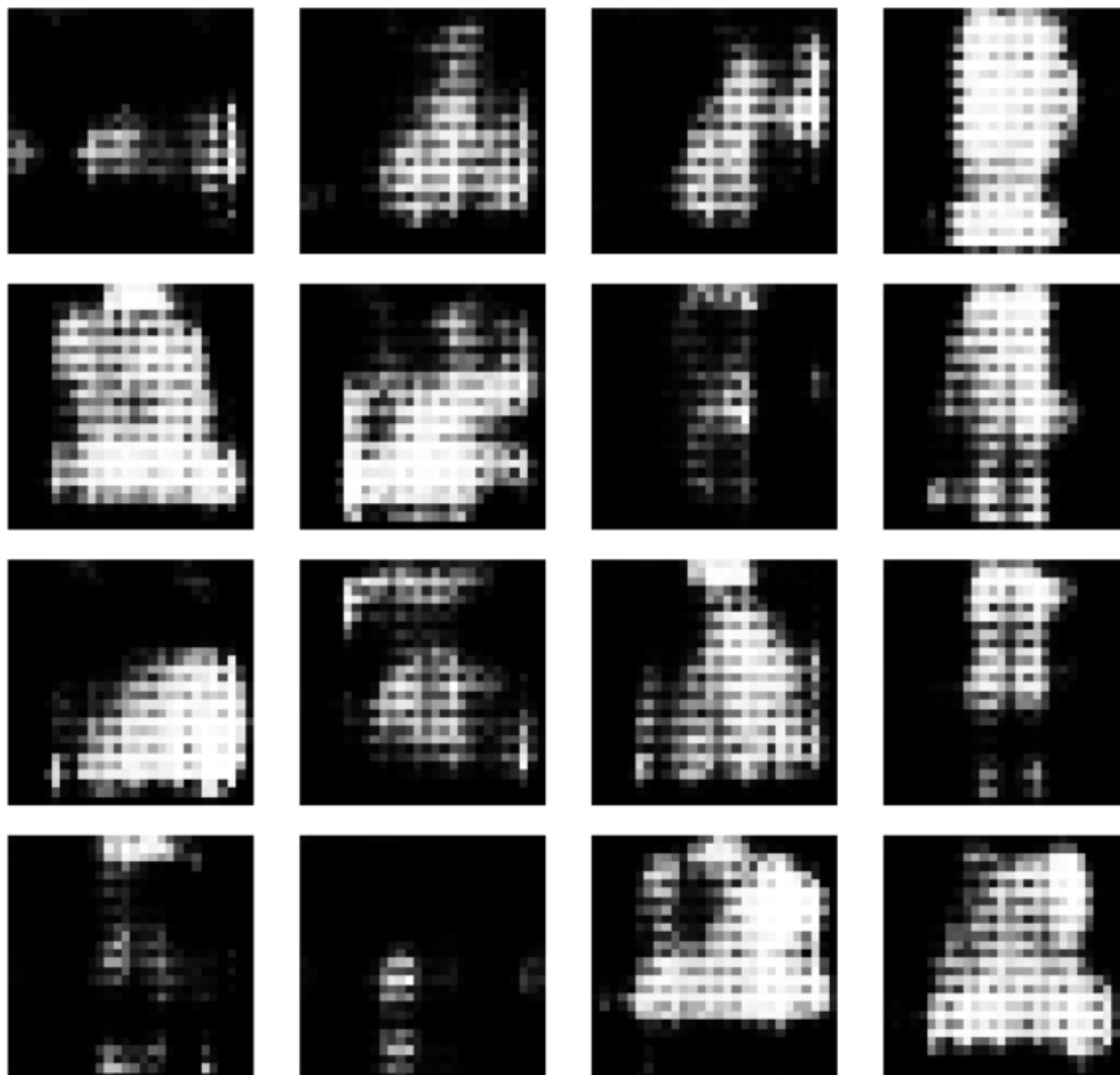
Epoch 3/10 | Generator Loss: 0.6803 | Discriminator Loss: 1.5707
Epoch 4/10 | Generator Loss: 0.7143 | Discriminator Loss: 1.3609

Generated Fashion-MNIST Images



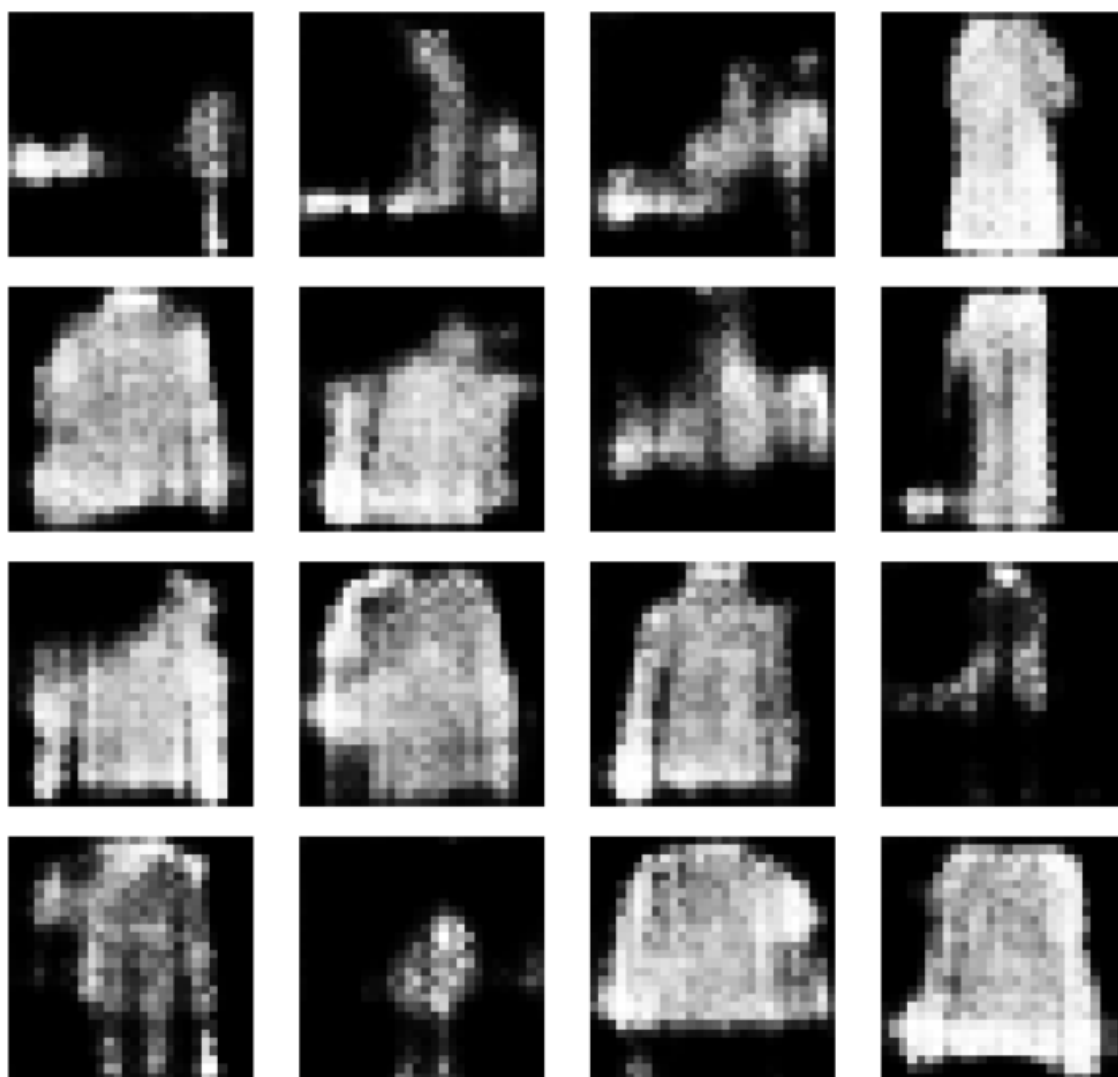
Epoch 5/10	Generator Loss: 0.7726	Discriminator Loss: 1.9998
Epoch 6/10	Generator Loss: 0.7753	Discriminator Loss: 1.2795

Generated Fashion-MNIST Images



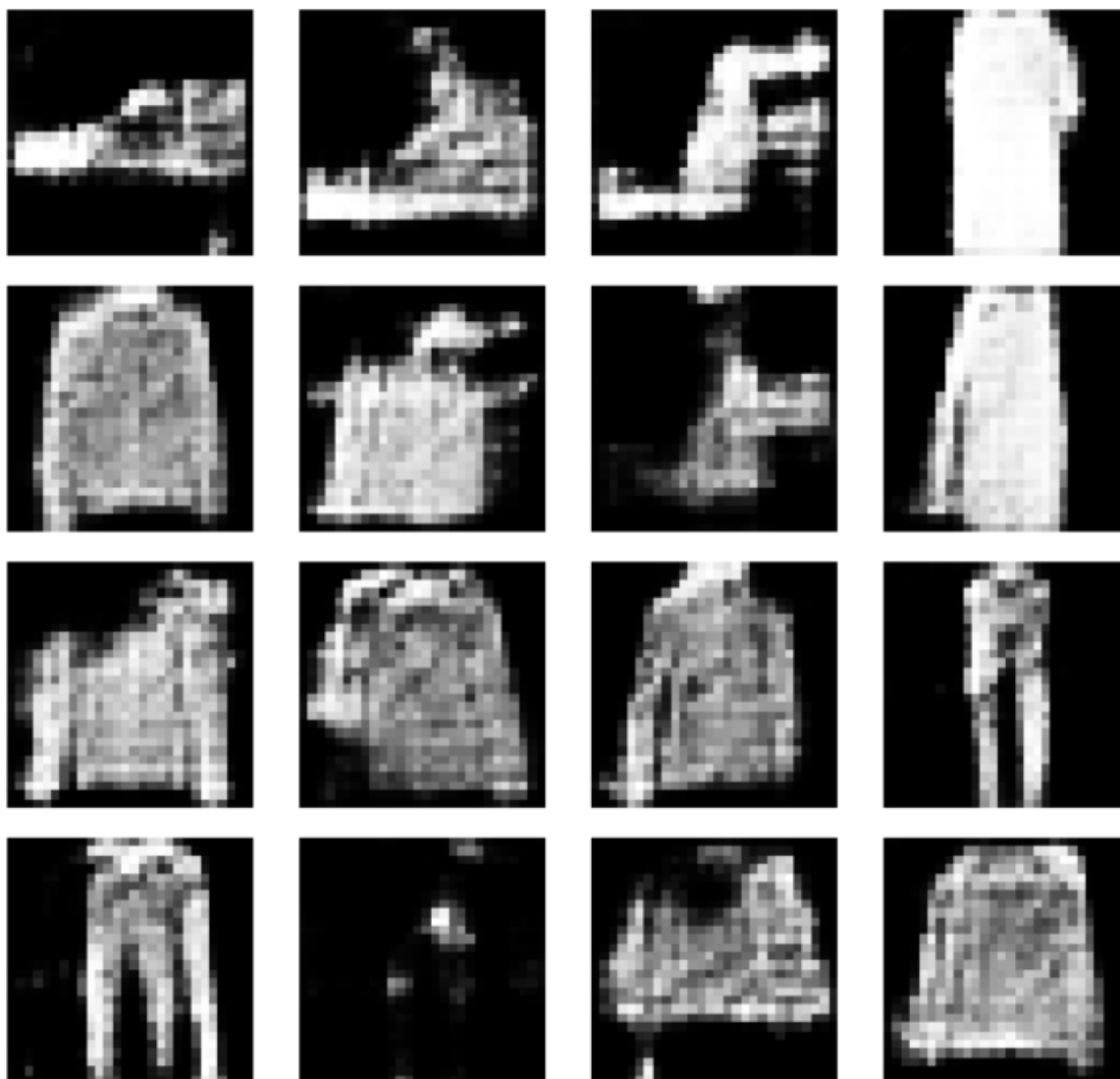
Epoch 7/10 | Generator Loss: 0.8535 | Discriminator Loss: 1.2061
Epoch 8/10 | Generator Loss: 0.8953 | Discriminator Loss: 1.1683

Generated Fashion-MNIST Images



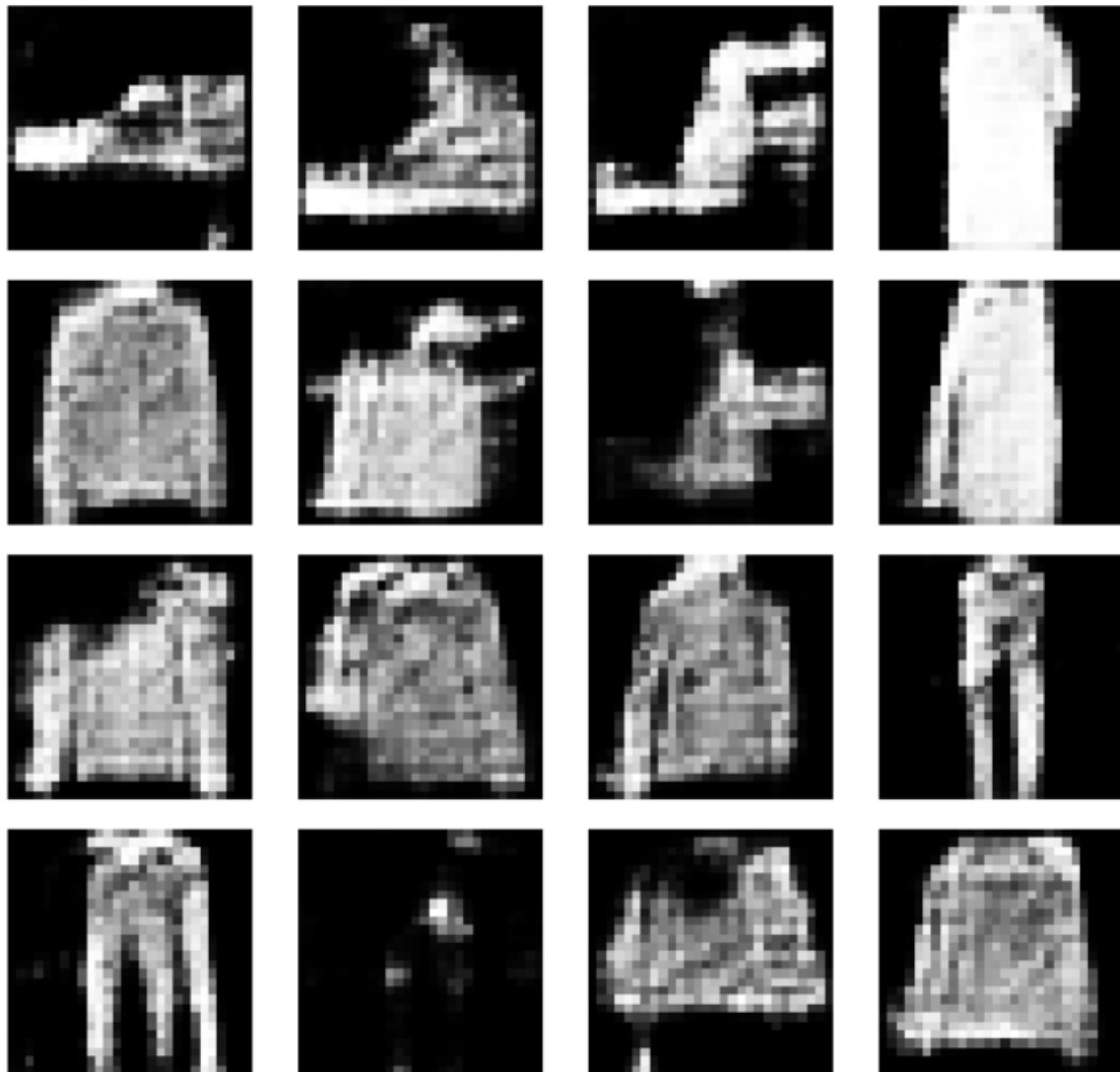
Epoch 10/10 | Generator Loss: 0.8711 | Discriminator Loss: 1.2335

Generated Fashion-MNIST Images



Training Completed!

Generated Fashion-MNIST Images



RESULT:

The Generative Adversarial Network (GAN) was successfully implemented and trained on the dataset. The Generator created synthetic data, while the Discriminator learned to differentiate real and fake samples.

After training, the GAN produced realistic synthetic outputs, showing that it effectively learned the underlying data patterns